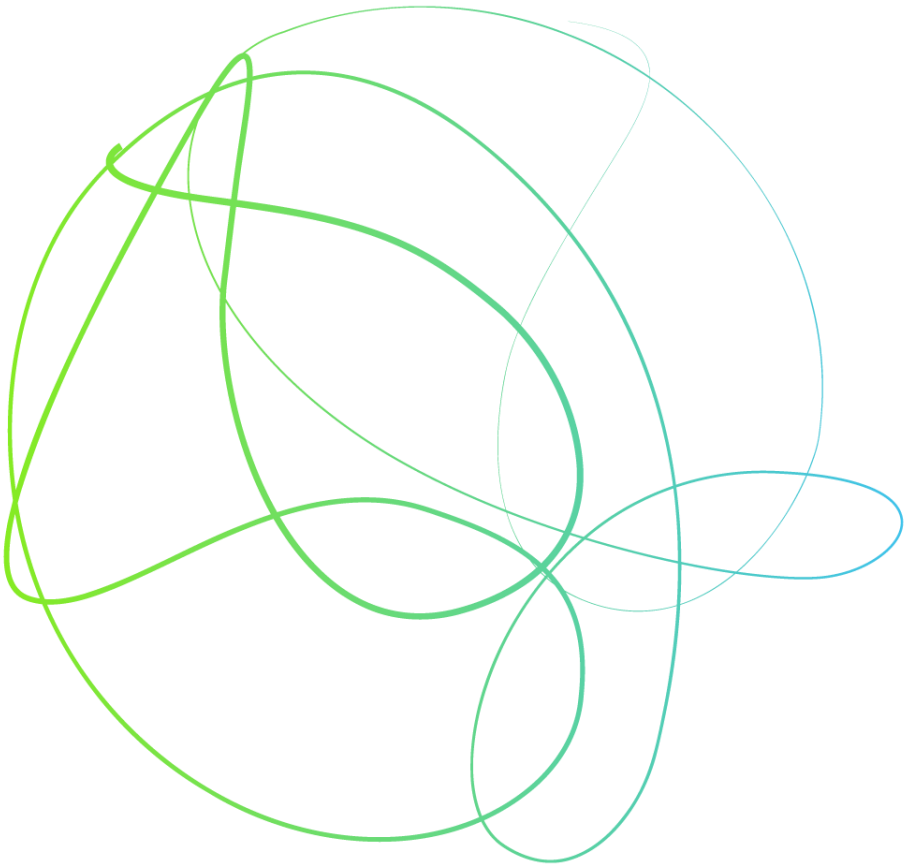
Deloitte.



Deployment and Management of Java Full-Stack Applications on AWS Cloud

Version Dated: 06.2026

Table of Contents

Problem Statement 3

Purpose 3

Objective 3

Tools and Technologies 3

Functional Requirements 4

Milestone 1: Manual Deployment of Java Full Stack Project using S3 bucket, EC2 and RDS, and Lambda 4

Overview 4

Architecture 4

Tasks 4

Milestone 2: Containerized Deployment of Microservice(s) on AWS 5

Overview 5

Architecture 5

Tasks 5

Non-Functional Requirements 6

Guidelines 6

Best Practices 7

Milestones and Evaluation Criteria 7

Copyright © 2026 Deloitte Development LLC. All rights reserved.

Problem Statement

Purpose

The purpose of this case study is to execute and demonstrate two deployment strategies for a Java full-stack application on AWS:

Milestone 1 - Manual deployment of the Java full-stack application, which includes setting up the frontend on Amazon S3, deploying backend Spring Boot microservices on EC2, configuring a MySQL database on RDS, integrating AWS Lambda for event-driven processing, and enabling monitoring with CloudWatch.

Milestone 2 - Containerized deployment approach, where backend microservice(s) is/are containerized using Docker, images are managed in Amazon ECR, and deployment is orchestrated via ECS with Fargate. This milestone also includes database integration with RDS, monitoring with CloudWatch, and leverages AWS managed services.

Objective

The objective is to perform manual and containerized deployments for Java full-stack projects on AWS. This will highlight the key implementation steps and AWS services used and assess the effectiveness, efficiency, scalability, and maintainability of each method. The aim is to guide best practices for cloud-native application deployment on AWS.

Tools and Technologies

Spring Boot	Framework for building microservices
Spring Cloud	Enables microservices architectures by providing features such as service registration and discovery and API gateway capabilities for routing and load balancing between services.
MySQL	Relational database for storing service data
Angular/React	A JavaScript framework or library used for developing frontend single-page applications (SPAs).
Amazon EC2	Hosts backend microservices and supporting services
Amazon ECS (Fargate)	Runs containerized microservices for scalable deployment
Amazon S3	Hosts static frontend assets (Angular/React)
Amazon RDS	Managed MySQL database service
Amazon ECR	Stores Docker images for ECS deployments
AWS Lambda	Runs serverless microservices or functions
AWS CloudWatch	Provides monitoring, logging, and alerting for AWS resources
Swagger	API documentation and visualization tool
Thunder Client (VS Code plugin)	API testing and validation tool

Copyright © 2026 Deloitte Development LLC. All rights reserved.

Functional Requirements

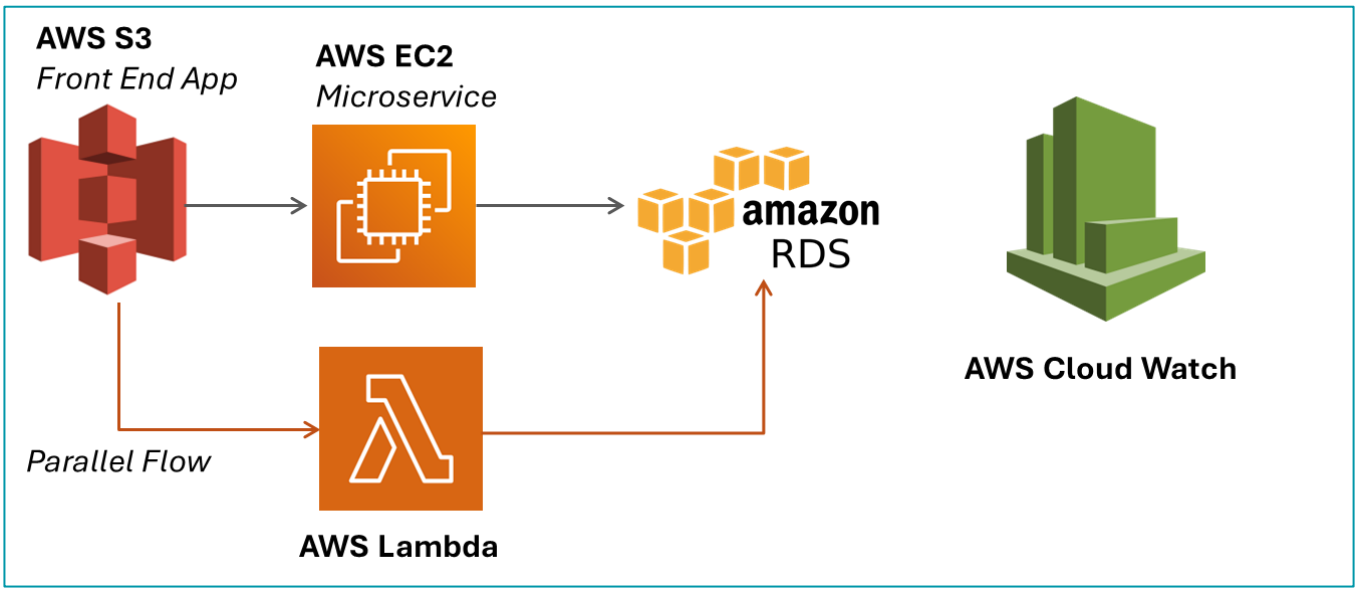
Let's discuss the details of each milestone, including the corresponding architecture and completion tasks for AWS-based deployment of Java full-stack projects.

Milestone 1: Manual Deployment of Java Full Stack Project using S3 bucket, EC2 and RDS, and Lambda

Overview

This milestone focuses on the manual deployment of a Java full-stack application using AWS services (S3, EC2, RDS). The deployment includes setting up the frontend, backend microservices, database, and monitoring, as well as integrating AWS Lambda for event-driven processing.

Architecture



Tasks

A. Set Up Frontend on Amazon S3

I. Upload static frontend assets (HTML, CSS, JS) to an S3 bucket.
II. Configure S3 bucket permissions for public access (if required).
III. Enable static website hosting on S3.

B. Deploy Backend Microservices on EC2

I. Provision EC2 instances for each microservice.
II. Install Java on each EC2 instance.
III. Deploy the Spring Boot JAR/WAR files to the respective instances.
IV. Configure security groups to allow necessary inbound/outbound traffic.

C. Set Up Amazon RDS (MySQL)

I. Launch an Amazon RDS instance with MySQL.
II. Configure database schema and user permissions.
III. Update backend microservice configurations to connect to the RDS instance.

D. Integrate AWS Lambda for Event-Driven Tasks

Copyright © 2026 Deloitte Development LLC. All rights reserved.

I. Create Lambda functions for specific event-driven processes (e.g., processing S3 uploads).
II. Set up S3 event triggers to invoke Lambda functions as needed.
III. Grant necessary permissions for Lambda to access S3 and RDS.

E. Configure AWS CloudWatch for Monitoring

I. Set up CloudWatch to collect logs and metrics from EC2 instances and Lambda functions.
II. Create dashboards and alarms for proactive monitoring (Optional).

F. Testing and Validation

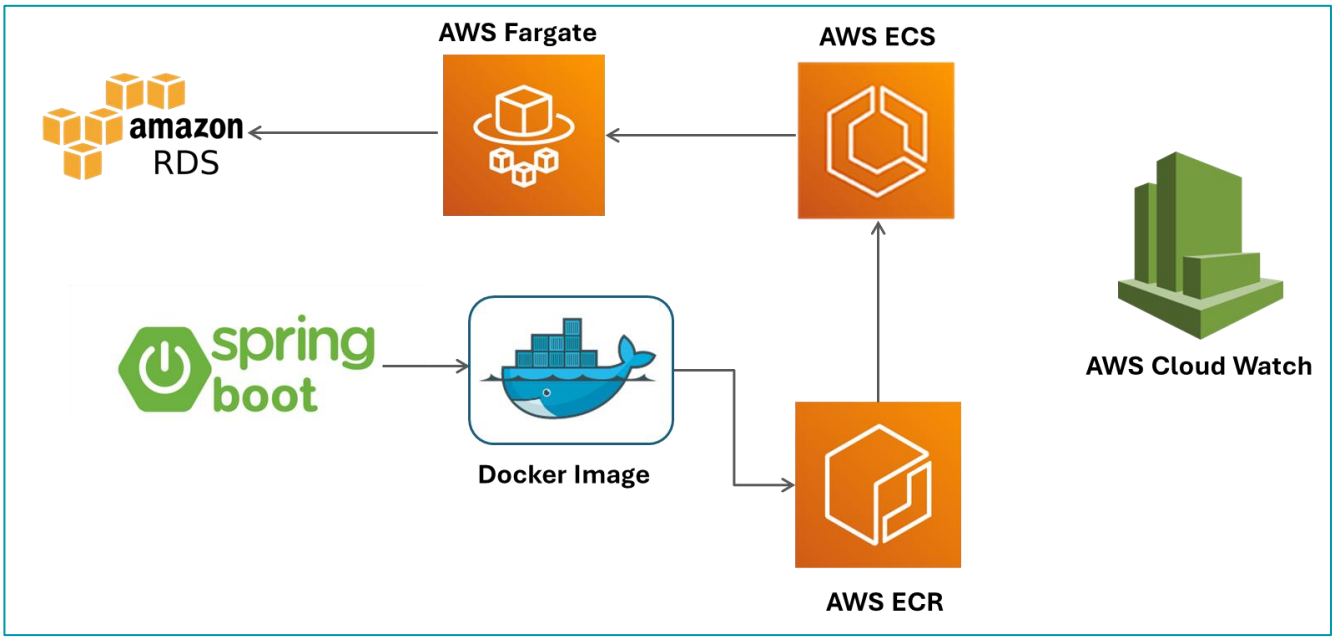
I. Test end-to-end functionality from the user browser through to the backend and database.
II. Validate Lambda event triggers and processing.
III. Monitor logs and metrics in CloudWatch for any issues.

Milestone 2: Containerized Deployment of Microservice(s) on AWS

Overview

This milestone focuses on containerizing Spring Boot Microservice deploying it on AWS using Docker, Amazon ECR, and ECS/Fargate. The deployment includes setting up the frontend, backend microservices as containers, database, and monitoring, leveraging AWS managed services for scalability and maintainability.

Architecture



Tasks

A. Containerize Backend Microservices (Spring Boot)

I. Create Dockerfile for each Spring Boot microservice.
II. Build Docker images for each microservice locally.
III. Test containers locally to ensure correct operation.

B. Push Docker Images to Amazon ECR

Copyright © 2026 Deloitte Development LLC. All rights reserved.

I. Create repositories in Amazon Elastic Container Registry (ECR) for each microservice.
II. Authenticate Docker CLI with ECR.
III. Push the built Docker images to the respective ECR repositories.

C. Deploy Containers on Amazon ECS/Fargate

I. Set up an ECS cluster using Fargate as the launch type.
II. Define ECS task definitions for each microservice, referencing the images in ECR.
III. Configure service discovery, networking, and security groups.
IV. Deploy and run the microservices as ECS services.

D. Set Up Amazon RDS (MySQL)

I. Launch an Amazon RDS instance with MySQL.
II. Configure database schema and user permissions.
III. Update ECS task definitions/environment variables to connect microservices to the RDS instance.

E. Configure AWS CloudWatch for Monitoring

I. Set up CloudWatch to collect logs and metrics from ECS tasks and services.
II. Create dashboards and alarms for proactive monitoring (Optional).

F. Testing and Validation

I. Test end-to-end functionality from the user browser through to the backend and database.
II. Validate container orchestration on ECS/Fargate.
III. Monitor logs and metrics in CloudWatch for any issues.

Non-Functional Requirements

- The application should have low latency and high throughput.
- The application and data should be secured by Authentication.
- System should be scalable and maintainable.

Guidelines

- Make use of best coding practices.
- You can make assumptions and incorporate additional features or functionalities.
- A simple but working prototype is preferred over a non-working but good-looking application.
- Add relevant comments for documenting deployment steps, configurations, and architecture for easy reference and onboarding.
- There would be a heavy focus on code quality and robustness of code in general.
- Use separate development, testing, and production environments to avoid conflicts and ensure stability.
- Assign least-privilege IAM roles and security groups to all AWS resources.
- Implement regular backups for databases and critical data.
- Leverage only Deloitte GitHub enterprise wherever it is needed.
- Use Git to maintain project repository, manage pull requests, and resolve conflicts.

Copyright © 2026 Deloitte Development LLC. All rights reserved.

Best Practices

Clear and Consistent Resource Naming	Use clear and consistent naming conventions for AWS resources, services, and code repositories.
Containerization	Package microservices as Docker containers for portability and scalability.
Service Discovery	Use Eureka or AWS-native solutions for dynamic service registration and discovery.
API Gateway	Centralize API management with Spring Cloud Gateway for routing, security, and monitoring.
Monitoring and Logging	Leverage AWS CloudWatch for real-time monitoring, centralized logging, and alerting.
Scalability	Design microservices and infrastructure to support horizontal scaling (EC2, ECS, Lambda).
Testing	Use tools like Thunder client (VS Code) for thorough API testing and validation before production releases.
Documentation	Integrate Swagger for up-to-date, interactive API documentation.

Milestones and Evaluation Criteria

S. No.	Details
Milestone 1	Set up Frontend app on Amazon S3
	Deploy Backend Microservices on EC2
	Set Up Amazon RDS (MySQL)
	Integrate AWS Lambda for Event-Driven Tasks
	Configure AWS CloudWatch for Monitoring
	Test and Validate End-to-End functionality
Milestone 2	Containerize Backend Microservices (Spring Boot)
	Push Docker Images to Amazon ECR
	Deploy Containers on Amazon ECS/Fargate
	Set Up Amazon RDS (MySQL)
	Configure AWS CloudWatch for Monitoring
	Test and Validate End to End functionality

Copyright © 2026 Deloitte Development LLC. All rights reserved.

Milestone 3	Push the build to your Github repository
Technical Interview	Discussion of Assumptions, Functionalities, and Validations

Copyright © 2026 Deloitte Development LLC. All rights reserved.

Deloitte.

About Deloitte
Deloitte refers to one or more of Deloitte Touche Tohmatsu Limited, a UK private company limited by guarantee ("DTTL"), its network of member firms, and their related entities. DTTL and each of its member firms are legally separate and independent entities. DTTL (also referred to as "Deloitte Global") does not provide services to clients. In the United States, Deloitte refers to one or more of the US member firms of DTTL, their related entities that operate using the "Deloitte" name in the United States and their respective affiliates. Certain services may not be available to attest clients under the rules and regulations of public accounting. Please see www.deloitte.com/about to learn more about our global network of member firms.

This communication contains general information only, and none of Deloitte Touche Tohmatsu Limited ("DTTL"), its global network of member firms or their related entities (collectively, the "Deloitte organization") is, by means of this communication, rendering professional advice or services. Before making any decision or taking any action that may affect your finances or your business, you should consult a qualified professional adviser.

No representations, warranties or undertakings (express or implied) are given as to the accuracy or completeness of the information in this communication, and none of DTTL, its member firms, related entities, employees or agents shall be liable or responsible for any loss or damage whatsoever arising directly or indirectly in connection with any person relying on this communication. DTTL and each of its member firms, and their related entities, are legally separate and independent entities.

Copyright © 2026 Deloitte Development LLC. All rights reserved.